Ingenieria de software 1

Jaime Nazar Anchorena

Contents

Trabajo Practico Especial **3**

Introduccion **3**

Unidad 1: Ciclos de vida **3**
- Que es . . . 3
- Por que . . . 3
- Modelos de ciclos de vida . . . 5
  - Cascada . . . 5
  - Incremental . . . 5
  - Evolutivo . . . 5
  - Prototipado . . . 6
  - Espiral . . . 8
- Proceso unificado . . . 8
  - Esfuerzo . . . 9
  - Mejores practicas . . . 9
  - Errores comunes . . . 10

Unidad 2: Extreme Programming **10**
- Agile manifeso . . . 10
- Extreme programming . . . 10
  - Valores . . . 11
  - Practicas . . . 11
  - Lifecycle . . . 12
  - Resuelve problemas de software . . . 12
  - Agile benefits . . . 12

Unidad 3: Metodologias - Scrum **13**
- Waterfall VS Agile(Scrum) . . . 13
- Scrum roles . . . 14
- Ceremonies . . . 14
- Artifacts . . . 14

Clase 3: Ingenieria de requerimientos **15**
- Que es un requerimiento . . . 15

Tipos clasicos . . . 15
Funcionales . . . 16
No funcionales . . . 16
Actividades de Ingenieria de Requerimientos. . . . 16
Stakeholders . . . 17
Riesgos de una mala gestion stakeholders . . . 17
Tenicas para identificar stakeholders . . . 17
Mapa de stakeholders . . . 18
Buenas practicas . . . 19
Extraccion de requerimientos . . . 19
Porque es dificil . . . 19
Tecnicas principales . . . 20
Buenas practicas . . . 20
Caracteristicas de requerimientos efectivos . . . 21
Analisis de requerimientos . . . 21
Caracteristicas de un buen conjunto de requerimientos . . . 21
Validacion de requerimientos . . . 21
Buenas practicas . . . 21
Gestion de requerimientos . . . 22
Trazabilidad de requerimientos . . . 22
Priorizacion de requerimientos . . . 22
Gestion de cambios . . . 22
Vision alternativa: Requerimientos como hipotesis . . . 22
TL;DR . . . 22

User stories **23**
Objetivos . . . 23
Espectativas vs realidad . . . 23
User stories exitosas . . . 23

Trabajo Practico Especial

- Vamos a tener que investigar bastante
- Promueven el uso de IA, para escribir codigo y para elegir
- Equipos de 7
- Nosotros vamos a tener que elegir la tecnologia a usar
- El alcance debe ser ambicioso, que nos esforcemos mucho
- Hay que saber que estamos resolviendo(porque, para quien)
- Vamos a aplicar metodologias agile, scrum
- Poner bara alta

Introduccion

Nosotros no buscamos la solucion optima teorica, sino buscamos que podemos hacer con nuestras limitacion de tiempo y economica. La ingenieria de software tiene dos dimensiones importantes, tiempo y escala(cuantas personas necesitamos para desarrollar una solucion y como se van a mantener los equipos).

Definicion interesante(suena mejor en ingles): Desarrollo entre multiples personas de programas de multiples definiciones. *Multi-person development of multi-version software.* Es decir, debe ir evolucionando, nunca se queda como esta ni se descarta y se empieza de nuevo.

Unidad 1: Ciclos de vida

Que es

Proceso sistematico y estandarizado por el cual un producto de software es desarrollado y mantenido a lo largo del tiempo.

Define un marco común para los procesos del ciclo de vida de los programas informáticos, con una terminología bien definida, a la que pueda remitirse la industria del software. Contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, el funcionamiento, el mantenimiento o la eliminación de sistemas, productos y servicios informáticos. Estos procesos del ciclo de vida se llevan a cabo mediante la participación de los interesados, con el objetivo final de lograr la satisfacción del cliente".

Por que

Sirve para contruir **software de alta calidad**, ese el el unico motivo. Esto tambien implica:

- **Predecir la etapa del ciclo de desarrollo**: Licencias, oficinas, personal, equipos, telefonos etc. Todo lo que necesitemos.
- Minimizar errores
- Minimizar costos
- **Reutilizable y facilmente mantenible**: Al aplicar determinados mecanismos y tecnologias vamos a poder hacer el software mas reutilizable y mantenible, entonces, menos costoso.
- Cumplir con los rquisitos del cliente
- Escalabilidad

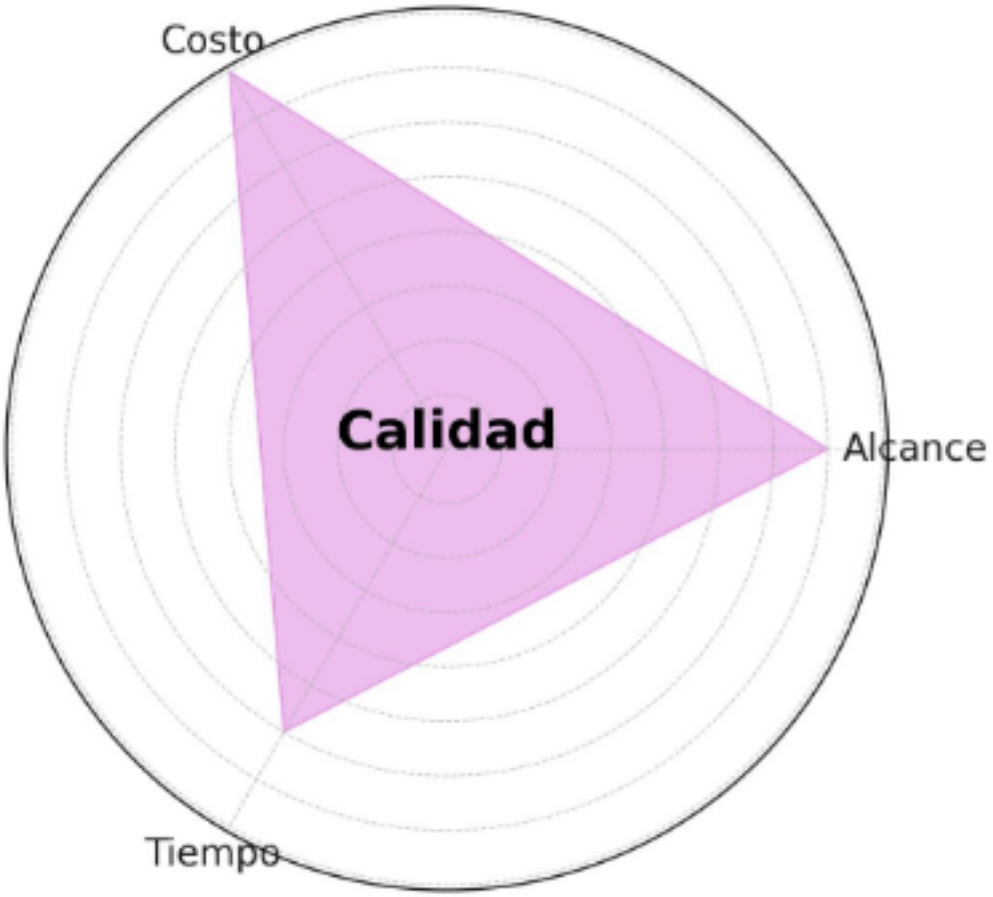

Figure 1: Representacion de la calidad de un software

Modelos de ciclos de vida

Todos los modelos tienen sus pros y sus contras, de manera que dependera de la naturaleza del projecto cual sera el modelo a utilizar.

Cascada

La finalizacion de una etapa implica en inicio de la siguiente. Las etapas son:

1. Analysis
2. Requirements specification
3. Design
4. Development
5. Testing and integration
6. Implementation/Deployment

Si me doy cuenta que, por ejemplo, durante la etapa de **testing** nos damos cuenta que hay que introducir un nuevo requerimiento, va a haber que volver a empezar(volver al analisis).

Ademas, lo primero de software que vera el cliente sera en la etapa final, lo cual puede ser meses o anos despues. Esto introduce el problema de que el cliente puede no estar satisfecho con el resultado final.

Incremental

Plantea resolver el problema principal del modelo cascada, trata de mostrar software funcionando lo antes posible.

Primero se define el sitema copleto y se lo divide en funcionalidades, luego, cada incrmento sigue todas las fases del ciclo de vida (análisis -> diseño -> codificación -> pruebas -> entrega).

Se entrega al usuario **funcionalidad completa por partes**.

Por ejemplo: Sobre Gmail: Google Meet, Google Classroom, Google Jamboard.

Evolutivo

Busca recoger **feedback** rapidamente, para poder mejorar la implementacion. Con cada ciclo se entregan diferentes versiones. Como funciona:

- Se define un conjunto minimo de requerimientos
- Se desarrolla una primera version o prototipo
- Se entrega al usuario y se recoge feedback
- Se planifican mejoras y se desarrolla una nueva version
- El ciclo se repite hasta alcanzar el producto final

Hay dos tipos de prototipos, el desechable(Tener cuidado con las expectativas del cliente, puede ser que no le guste que este *atado con alambre*) y el que es simplemente una version del programa, en la cual se puede ir iterando.

Modelo de desarrollo que , a diferencia del de Prototipos, busca reemplazar el viejo sistema con uno nuevo que tendría la propiedad de satisfacer los nuevos requerimientos lo más rápido posible. El desarrollo evolutivo asume que los requerimientos están sujetos a cambios continuos y que la estrategia para enfrentar aquello pasa por un reflejo, también continuo de aquellos cambios.

Por ejemplo: Windows, Office, Ubuntu.

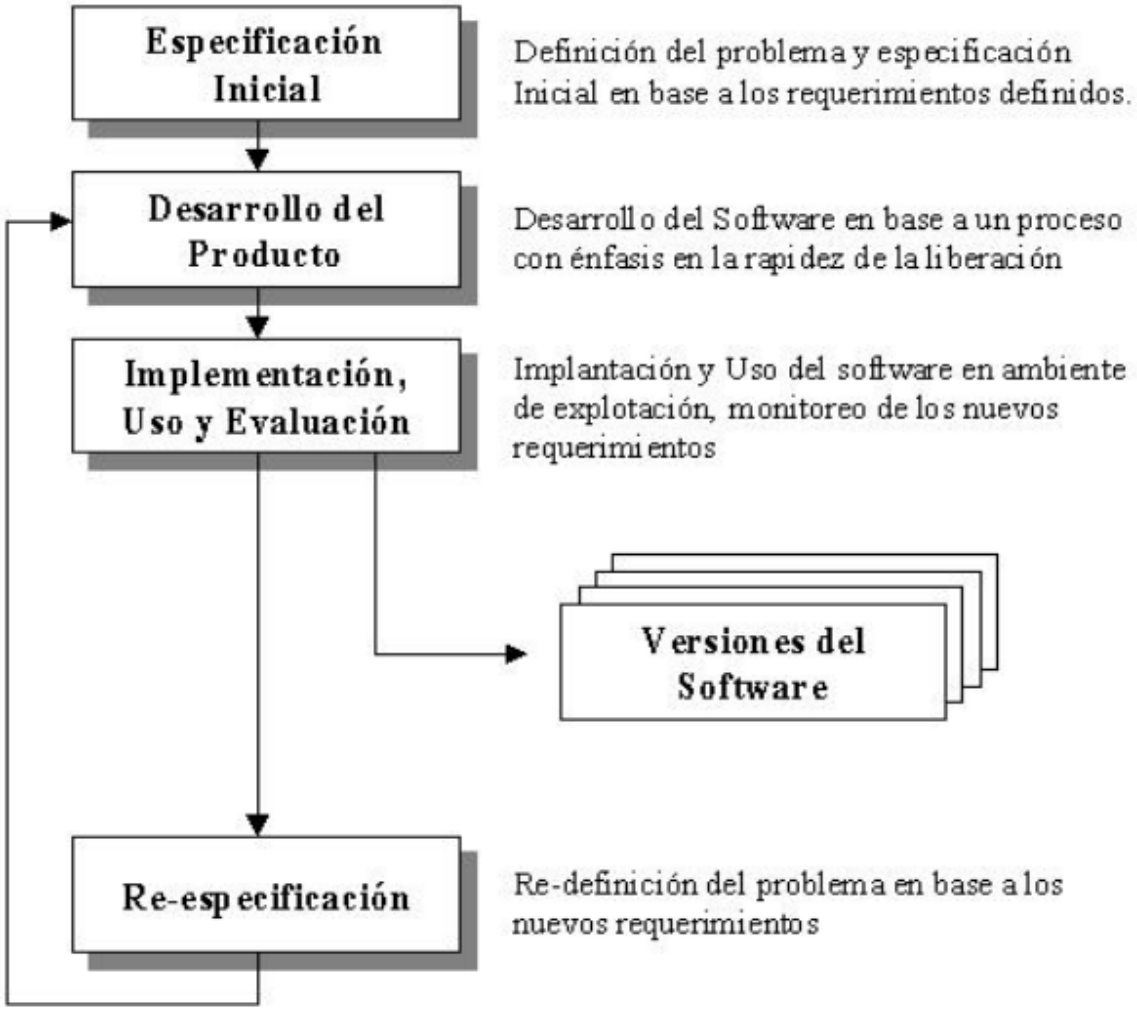

Figure 2: Modelo evolutivo

Nota: La pricipal falla de los proyectos informaticos es la falta de **comunicacion**. Esto es reponsabilidad del Ingeniero de Software, el prototipado busca resolver parte de esta comunicacion.

Prototipado

La idea es poder mostrar como pensas que puede ser el sistema. Modelo que pone énfasis en la etapa de Especificación de Requerimientos a través de la construcción de prototipos que aproximan al usuario a la idea final del sistema, con objeto de poder clarificar los requerimientos.

Sirve para poder descubrir la funcinalidad completa que quiere el cliente. Esto se debe a que el cliente generalmente no sabe muy bien lo que quiere, hay que validad lo mas posible si estamos desarrollando lo que quiere el cliente.

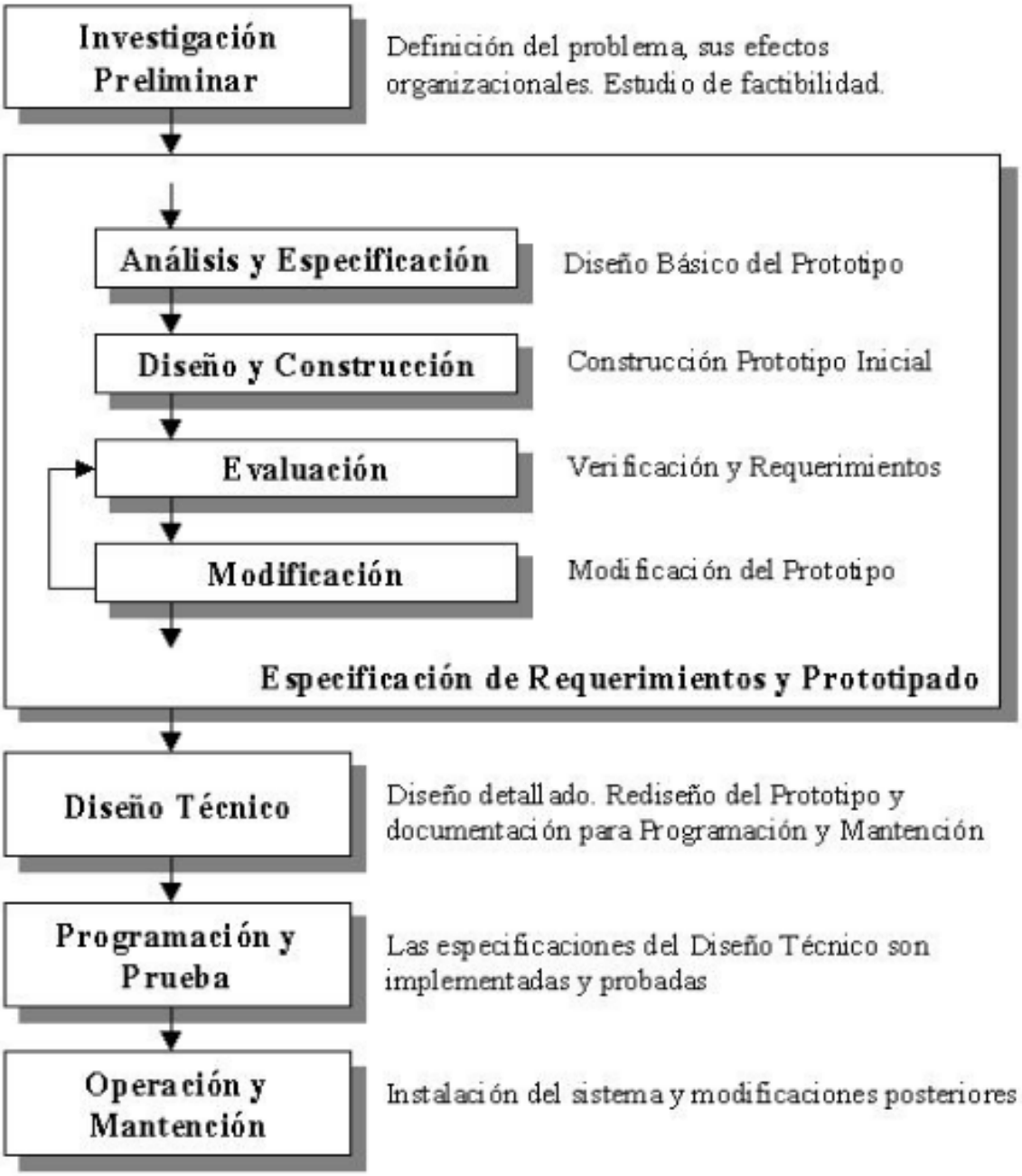

Figure 3: Modelo prototipado

Espiral

El mas completo, usa todos los anteriores:

- Prototipado
- Evolutivo
- Iterativo
- Incremental

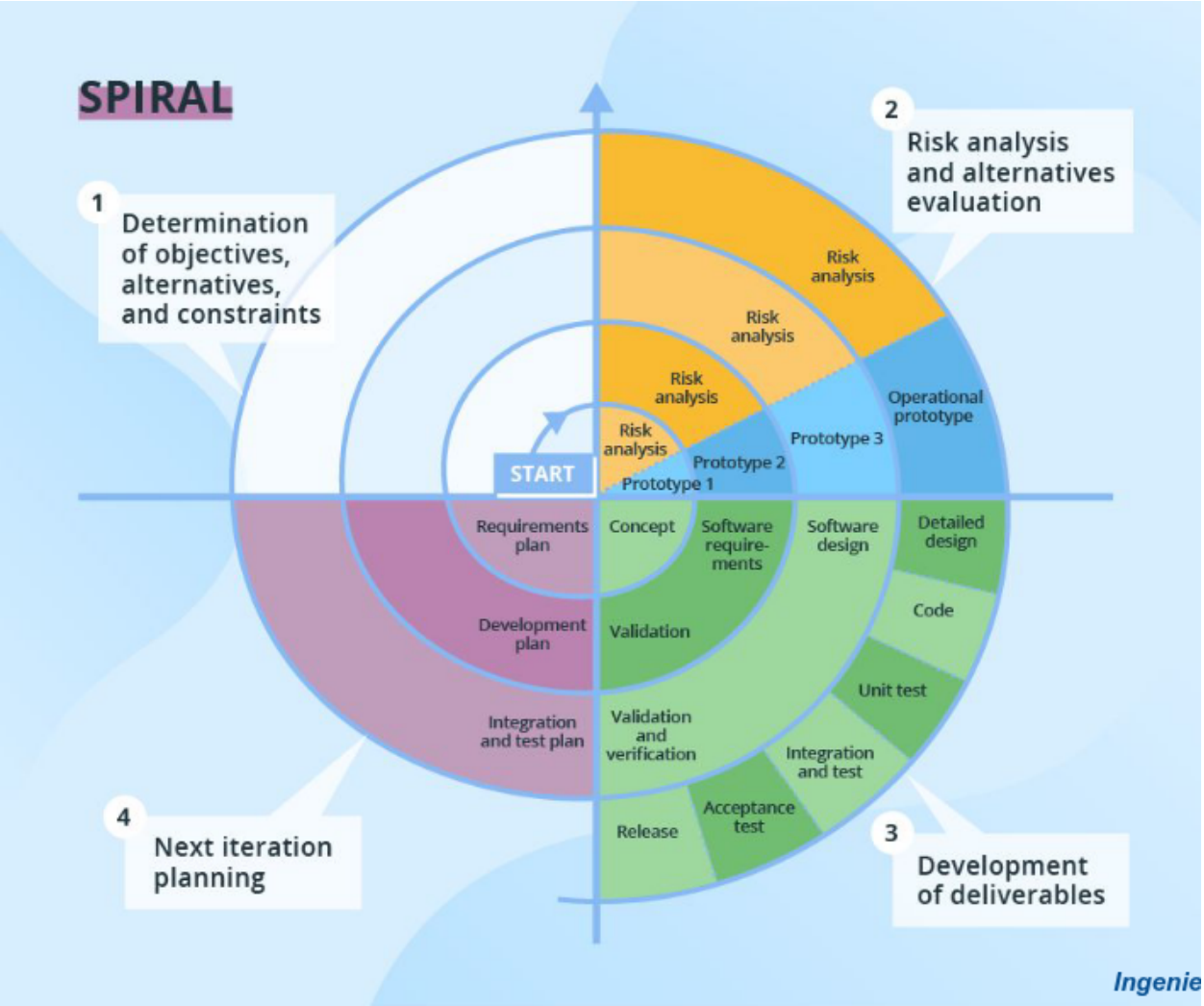

Figure 4: Modelo espiral

Una cosa muy importante que contempla este modelo es el analisis de riesgos, ademas de su gestion dentro de cada iteracion. Trata las falencias de los modelos anteriores.

Proceso unificado

De una manera es la madre de todos estos projectos. UP es un framework de proceso detallado, define aproximadamente 50 opcionales artefactors(non-software work products), tales como:

- Vision, Lista de riesgos, etc

- En RUP estan organizados dentro de las diciplinas, y los encontramos como templates(la documentacion)
- La regla a seguir con respecto a la creacion de estos documentos es “Menos es mejor”
- Esta unica adaptacion es llamada **Development Case** del projecto

Esto esta basado en projectos anteriores, tomando las practicas que funcionaron. No hace falta usar todos los artefactos en nuestros projectos. No quieren que agamos documentacion por documentacion, sino que tenga un *target*.

Esfuerzo

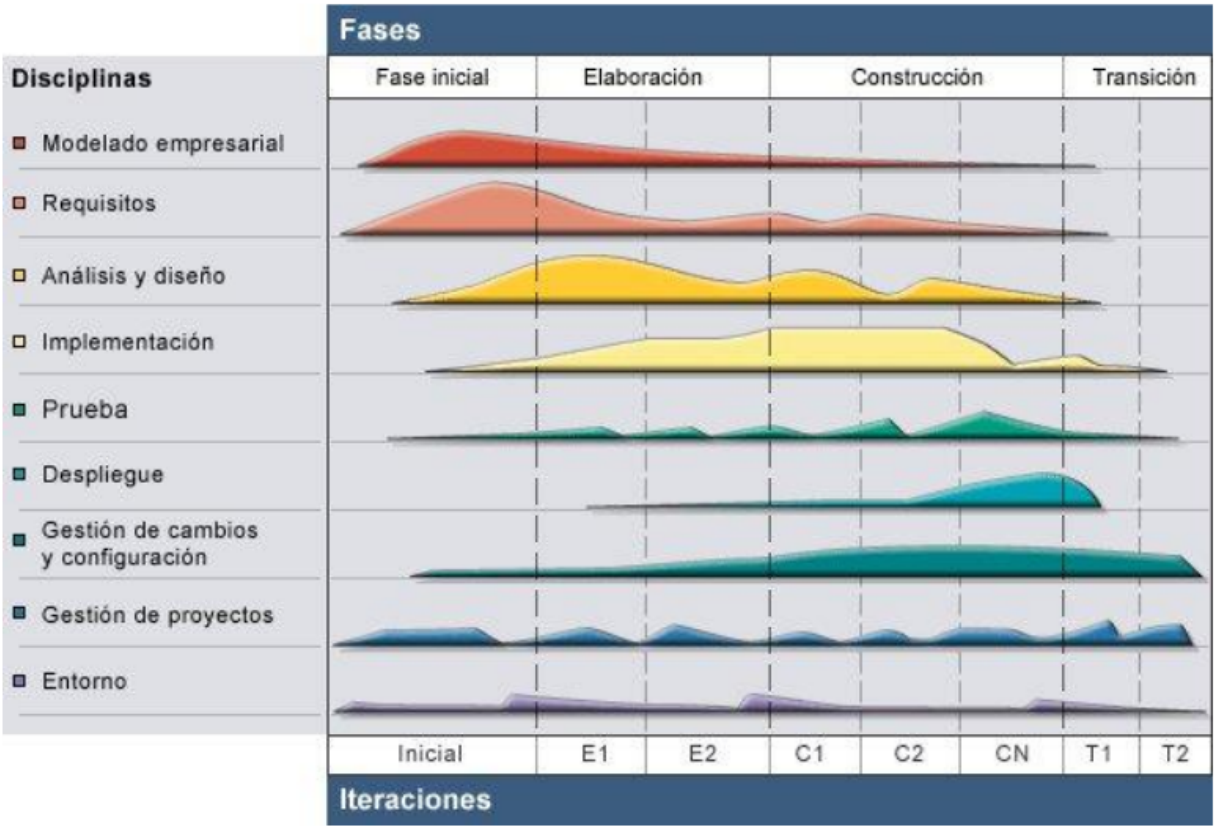

Figure 5: Actividad por diciplinas RUP

Los recursos y el staff son variables.

Mejores practicas

- **Iteraciones TimeBoxed**: Recomendadas entre 2 y 6 semanas, como hackatlon.
- **Programando los elementos de alto riesgo, y alto valor**: Incentivar a la reutilización de componentes
- **Verificación de calidad continuada**: Uso de técnicas de integración y testeo continuo, como XP
- **Modelado visual**: Esquemas en una pizarra durante 1 hora. Esquemas basados en UML

- **Administración de requerimientos**: Encontrar (req. Funcionales a través de workshops). Organizar (según prioridades, atributos, riesgos, ...) y seguir los requerimientos (según su estado – asignado, en curso, ...)
- **Administración del cambio**: A través de la administración de la config. y del control de versión. El uso de un protocolo de solicitud de cambios. Releases al final de cada iteración

Errores comunes

- **Iteraciones**: Demasiado largas(se pierde el enfoque). Cada iteración no finaliza con un release en producción
- **El objetivo de la fase elaboración**: Crear un prototipo desechable
- **Planeamiento**: Predictivo no adaptativo. Diferir el testeo mayor hasta cerca de finalizar el proyecto
- **Documentación**: Hacer cantidades de diagramas de UML. Realizar un "Development Case" demasiado complejo, con muchos work products. El Software Architecture Document (SAD) finalizado antes de terminar la elaboración

Unidad 2: Extreme Programming

El desarrollo de software presenta varios problemas:

- Entregas tardias y/o incompletas
- Cancelacion de proyectos
- Sistema desactualizado
- Alta tasa de errores
- Reglas de negocio sin entender
- Cambios de requerimientos
- Desercion de programadores
- Ambientes de alto grado de incertidumbre

Agile manifeso

Para solucionar esto se propone las siguientes reglas:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

  Nota: Habia anotado mas cosas, pero se perdio. Basicamente era un desarrollo de cada punto, pero se puede deducir.

Extreme programming

- Metodología liviana, eficiente de bajo riesgo, flexible, predecible y científica.
- Orientada a equipos pequeños o medianos.
- Para desarrollar software teniendo como premisa que los requisitos cambian rápidamente.
- Especialmente diseñadas para ambientes de gran incertidumbre.

Valores

Figure 6: Valores de XP

Practicas

Juego de planificacion Determinar rapidamente el aclance de la proxima release combinando las prioridades del negocio y las limitaciones tecnicas. Planificar como maximo dos releases por adelantado. Como la realidad supera la planificacion, actualizar el plan.

Releases pequenas Poner en produccion el sistema mas pequeno que funcione, luego agregar funcionalidad iterativamente. Cada release debe tener una unidad de coherencia.

Metaforas Guiar a todo el grupo de desarrollo con una historia simple de como debe funcionar el sistema. Ayudar a comprender los elementos basicos del sistema, debe ser refinada a medida que avanza el proyecto. Es una vista desde *10k metros* del sistema.

Diseno simple El sistema debe ser lo mas simple posible, remover toda complejidad. El diseño correcto 1. Debe pasar todos los testeos 2. No tiene lógica duplicada 3. Muestra la intención del programador 4. Tiene la menor cantidad posible de métodos y clases

El futuro es incierto, entonces no diseñar de más para una futura funcionalidad

Testing Los testeos unitarios deber ser parte del codigo. Los testteos de funcionalidades …

Refactoring Implica llevar el programa a la forma mas simple una vez que se introducen cambios, favorece el agregado de nuevos cambios. Si existe codigoduplicado debe refactorizarse.

Pair programming Una de las personas escribe el codigo y piensa en la mejor forma de hacerlos, la otra persona piensa de forma estrategica(Es correcta la solucion? Hace falta definir otro testeo? Hay forma de simplificarlo?). La pareja debe ir rotando para fomentar el conocimiento del sistema.

Compartir autoria Cualquiera que ve la oportunidad de agregarle valor a un codigo debe hacerlo, no hay una persona que sea duena de un bloque de codigo. Mejora los tiempos de correccion de bugs, se ve favorecida por la programacion de a pares.

Integracion continua El codigo debe ser integrado y testeado a intervalos regulares. Economicamente conviene tener una maquina dedicada a esta tarea, cualquier persona que agregue cambios a la release debe cerciorarse que el 100% de los testeos sean exitosos.

40hs semanales Trabajar más de esta cantidad de horas degrada la capacidad creativa del equipo. Esto repercute directamente en la cantidad de bugs presentes, se permite que una semana se trabaje más horas. Si esta situación se presenta en dos semanas consecutivas entonces el problemano puede solucionarse trabajando más horas

Cliente on-site El cliente debe estar con el equipo, disponible para responder cualquier duda que pueda surgir. Por cliente se entiende alguien que va a utilizar el sistema cuando el mismo se encuentre en producción. Mientras no es consultado, el cliente puede seguir realizando su trabajo.

Estandares de codificacion Se deben definir para facilitar la práctica de la autoría compartida de código. Deben tener como premisa hacer el mínimo trabajo posible Debe enfatizar la comunicación. El equipo debe adoptarlo de común acuerdo.

Lifecycle

Resuelve problemas de software

Prevee cuando se va a hacer algo mal, siempre corrijiendo.

> **Nota**: Hay mas en el apunte.

Agile benefits

- Better product quality
- Customer satisfaction

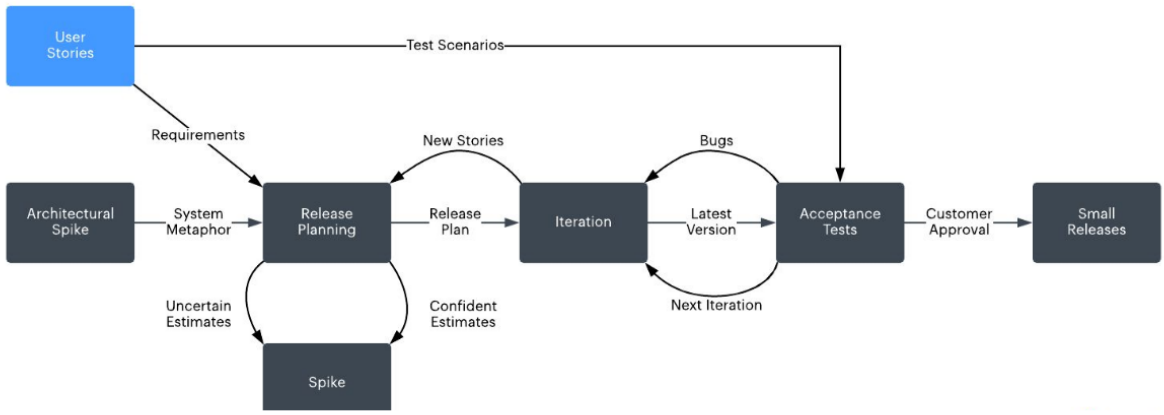

Figure 7: XP lifecyle

- More control on projects
- Risk reduction
- Faster ROI
- Faster time to market
- Higher productivity

Nota: XP es solo usado por el 1% de los programas de desarrollo.

Unidad 3: Metodologias - Scrum

La mas usada en la actualidad, framework ágil para construir, entregar y mantener productos complejos. Diseñado especialmente para equipos pequeños, altamente cohesionados. Todo el trabajo sucede en iteraciones cortas llamadas sprints. Es un estandar muy puro, generalmente se usa con alteraciones.

Hoy en dia esta metodologia se usa en otras areas que no son de software. Un sprint dura entre una semana y un mes, la catedra usa 15 dias.

Nota: En la practica nos van a pedir usar Scrum.

Waterfall VS Agile(Scrum)

Con agile se busca siempre tener una version funcional que ataca el problema(incluso si no es la version final), mientras el cliente da feedback, se reinicia el ciclo. Se planifica el sprint y se lo vuelve a presentar al cliente. Esto no se puede con waterfall, con waterfall se crea software no mantenible y se debe saber cual es el resultado final de antemano(puede ocurrir que no es lo que el cliente quiera).

Agile permite mostrar el proceso y lograr una cohecion entre lo que el cliente quiere y lo que le das.

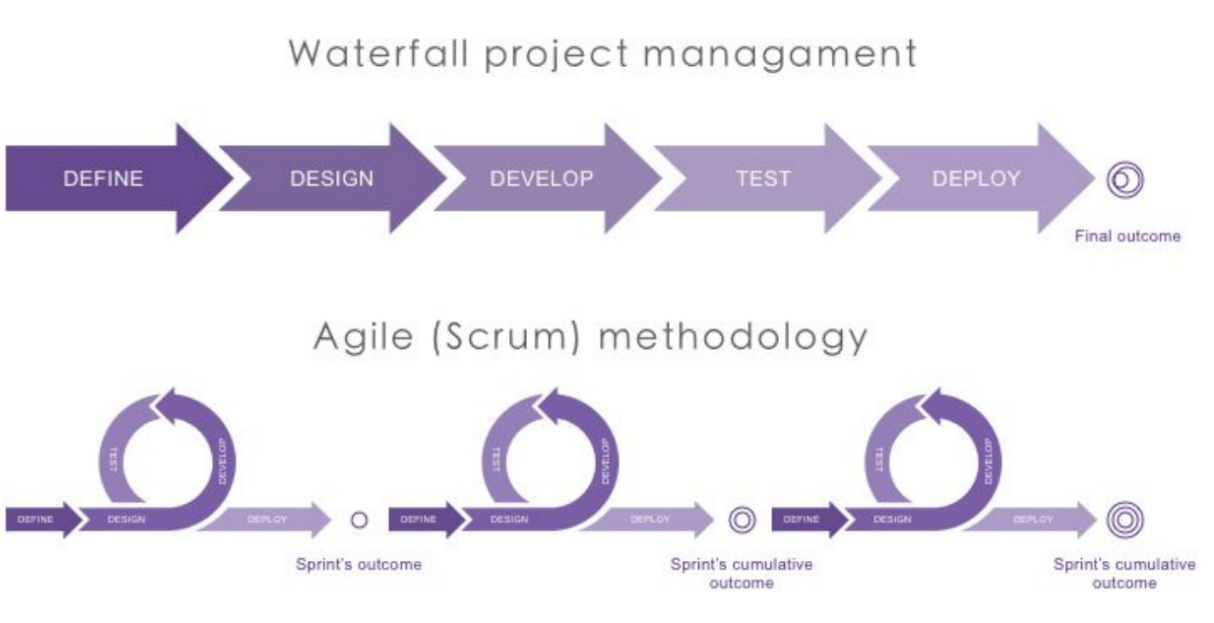

Figure 8: Waterfall VS Agile(Scrum)

Scrum roles

No hay gerarquia vertical(en la vresion pura) no hay jefe, sino que son todos iguales.

- **Product owner**: Se preocupa por los intereses del cliente.
- **Scrum master**: Se encarga de llevar adelante todas las ceremonias del scrum y documentar que se esta haciendo.
- **Scrum team members**: Desarrolladores
- **Stakeholders**: Personas externas al equipo itneresados en el producto.

  Nota: El manifesto indica que no deben ser mas de 10 personas por equipo, si son mas se debe dividir en dos equipos distintos.

Ceremonies

1. **Sprint planning**: To craft a sprint goal and plan to achieve it.
2. **Daily scrum**: To review team progress on sprint goal and adjust the plan to get there.
3. **Sprint review**: To review de work completed in the sprint with stakeholders.
4. **Sprint retrospective**: To rewview the product development process and identify areas for continuos improvements

Artifacts

Elementos uttilizados en el scrum para llevar adelante el projecto, en el backlog se lleva registro del proceso. Es una tarea que los desarrolladores van a tener que implementar en el codigo.

Cuando estan en el spring backlog, se estima el user story en base al esfuerzo que requiere realizar esa tarea. Se mide en story points, si el sprint empieza con 22, la idea es terminar en 0 el sprint. Si no llega a 0, ocurre

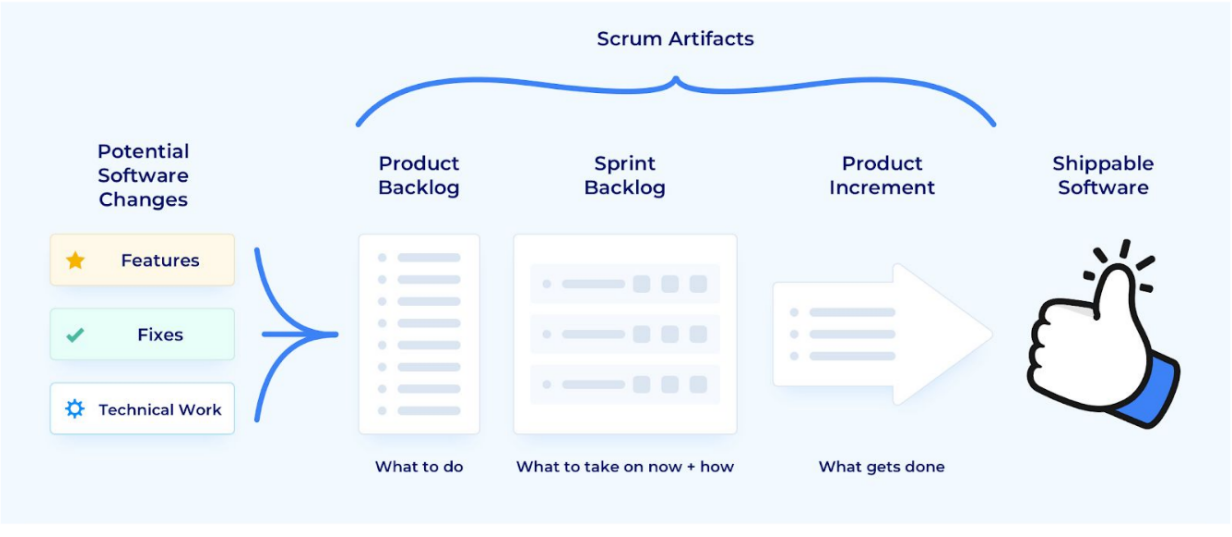

Figure 9: Scrum artifacts

un **carry over**, se lo pasa al siguiente spring y se debe analizar porque paso. Lo ideal es que el grafico tienda a una funcion lineal, el grafico representa story points en relacion al tiempo.

> **Nota**: Vamos a tener que hacer este tipo de graficos en nuestros trabajos.

Clase 3: Ingenieria de requerimientos

Conocer bien los requerimientos puede ser la diferencia entre un corto o un largo tiempo de desarrollo, es muy importante.

Que es un requerimiento

- Cualquier factor que afecte el diseno del sistema
- Condicion o capacidad que un sistema debe satisfacer para cumplir con una necesidad
- Especificacion(texto, dibujo, otro tipo de herramienta visual) de lo que debe ser implementado. Descriptores de como el sistema debe comportarse, o propiedad o atributo del sistema.
- Pueden ser una restriccion en el proceso de desarrollo del sistema.

A su vez, un requerimiento puede tener muchos subrequerimientos, por ejemplo, seguridad implica muchas funcionalidades pues esta compuesta por muchas areas. Comportamiento y propiedad no necesariamente son lo mismo, presentan diferencias.

Tipos clasicos

- **Funcionales**: Comportamientos, funciones, servicios. Las que se refieren por las parsonas comunes, como el cliente.
- **No funcionales**: Restricciones de calidad o condicionales del entorno(seguridad, expansibilidad, etc. cosas que terminen con -ad).
- **Dominio / restricciones**: Proviene del entorno o regulaciomnes externas. Depende de las normas de calidad(por ejemplo, algun ISO), industria, etc.

Funcionales

Se suelen representar como **user stories**, describe lo que el sistema debe hacer, es un comportamiento especifico del sistema.

Ejemplo: El sistema debe permitir que los usuarios se registren con email.

No funcionales

Define propiedad o restriccion, pero no un comportamiento especifico, son de los mas dificiles de definir y cumplir. Asociados generalmente a los **atributos de calidad**, son los mas importantes a la hora de disanar la arquitectura del sistema.

Algunos atributos de calidad son:

- Rendimiento
- Usabilidad
- Seguridad
- Mantenibilidad
- Portabilidad
- Interoperabilidad
- Confiabilidad
- Escabilidad

Su importancia dependera del contexto, puede ser que unos sean despreciables mientras otros sean sumamente importantes.

Esta clasificacion es util debido a que generalmente se suele utilizar en plantillas formales y documentacion tecnica. Tambien porque facilida la organizacion y tracibilidad, es requerida en auditoria, licitaciones, contratos. Y es util para roles especificos: tester, arquitectos, legales.

Sin embargo, la division entre funcional y no funcional es un poco **artificial**, no necesariamente son categorias distintas. Por ejemplo, si el sistema no es accesible, eso seria un requisito funcional para las personas que no puedan usar el sistema sin la accesibilidad, mientras que para otros que si, es no funcional.

Actividades de Ingenieria de Requerimientos.

Esto es secuencial, una vez se termina con una actividad se pasa a la siguiente.

1. **Extraccion o elicitacion**

Identificacion de requerimientos y stakeholders. Se utilizan tecnicas como:

- Entrevistas
- Workshops
- Observacion

2. **Analisis**

Comprension y organizacion de los requerimientos. Se utilizan tecnicas como:

- Modelado
- Prototipado
- Analisis de impacto

3. **Especificacion**

Redaccion formal de los requerimientos. Se utilizan tecnicas como:

- Plantillas
- Casos de uso
- User stories
- Diagramas

4. **Validacion**

Asegurar que los requerimientos reflejan las necesidades reales. Se utilizan tecnicas como:

- Revisiones
- Prototipos Pruebas de aceptacion

5. **Gestion**

Control de cambios, trazabilidad y priorizacion. Tecnicas:

- Herramientas de gestion de requerimientos
- Matrices de trazabilidad

Stakeholders

Personas o grupos que afectan o son afectadas por el sistema. No solo usuarios finales, involucra patrocinadores, soporte, legales, IT, etc. Cada uno tiene necesidaes y espectativas diferentes. Por lo tanto es muy importante recompilar informacion de cada uno.

- Internos vs Externos
- Operativos(usuarios directos)
- Gerenciales(responsables de decisiones y presupuesto)
- Tecnicos(IT, arquitectura, seguridad)
- Indirectos(legales, reguladores, soporte, marketing)

  Nota: Suele pasar que los sistemas se basan en necesidades gerenciales(no tienen experiencia directa), pero son inposible de usar para los usuarios.

Riesgos de una mala gestion stakeholders

Puede ocurrir que los requerimientos esten incompletos o sean contradictorios, en ese caso se debe decidir cual es mas importante. Tambien puede haber un cambio de alanze tardio debido a falta de comunicacion, o puede haber falta de validacion o aceptacion del producto. Finalmente, pueden realizarse decisiones basadas en suposiciones erroneas.

Tenicas para identificar stakeholders

- Entrevistas iniciales y mapeo organizacional.
- Stakeholders map, se realiza en base a la informacion obtenida en el punto anterior.
- Personas, tecnica donde se arman arquetipoc que resumen motivaciones y necesidades(perfil inventado).
- Workshops colaborativos, alinean espectativas.

Nota: Esta parte es sumamente importante, que en el fondo no son mas que requerimiento funcionales y no funcionales. Hacer las preguntas correctas.

Nota: No se enamoren de la solucion, sino que del problema.

Mapa de stakeholders

Tenemos que tener una conversacion muy fluida con los mas importante y mantener contacto y mantener al tanto a los menos importantes

Circulo Poner la solucion en el medio y luego agregar circulos concentricos. Permite ver quienes son los grupos de personas con los que se va a interactuar de manera mas fluida, pero tambien los que hay que mantener en contacto.

Figure 10: Circulo stakeholders

Esquema Ver figura.

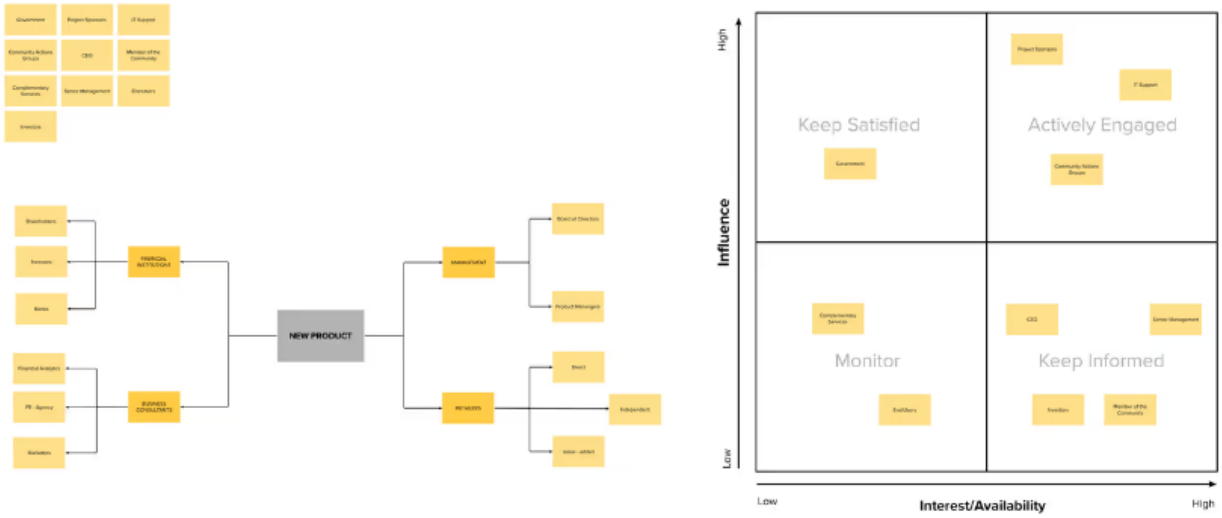

Figure 11: Esquemas stakeholders

Persona canvas Hay muchas maneras de hacerlo, depende del proyecto. Se pueden encontrar facilmente online y rellenar.

> **Nota**: Puede ser que no nos pidan usar estas herramientas, pero valoran mucho que las usemos, incluso que inventemos nuestro propio metodo.

Buenas practicas

- Involucrar a los stakeholders desde el inicio
- Documentar que se dijo(trazabilidad)
- Mantener una comunicación continua y adaptativa
- Gestionar conflictos de prioridades con transparencia

Extraccion de requerimientos

Es el proceso de descubrir, recopilar y entender los requerimiendos desde las partes interesantes. No se trata solo de recoger pedidos, sino de interpretar necesidades reales, a menudo tacicas o no articuladas.

Vale mucho mas la pregunta correcta que la especificacion completa y con lujo de detalle. Porque suelen pasar malentendidos en el caso contrario.

> **Nota**: Hosting, db, cloud(AWS, azure, etc.) cuentan como proveedores.

Porque es dificil

- Los stakeholders no siempre tienen claro lo que necesitan
- Expresan soluciones, no problemas (ej: “Quiero un botón para exportar a Excel” -> ¿para qué?)
- Uso de lenguaje ambiguo o contradictorio, sobre todo cuando entrevistamos personas distintas

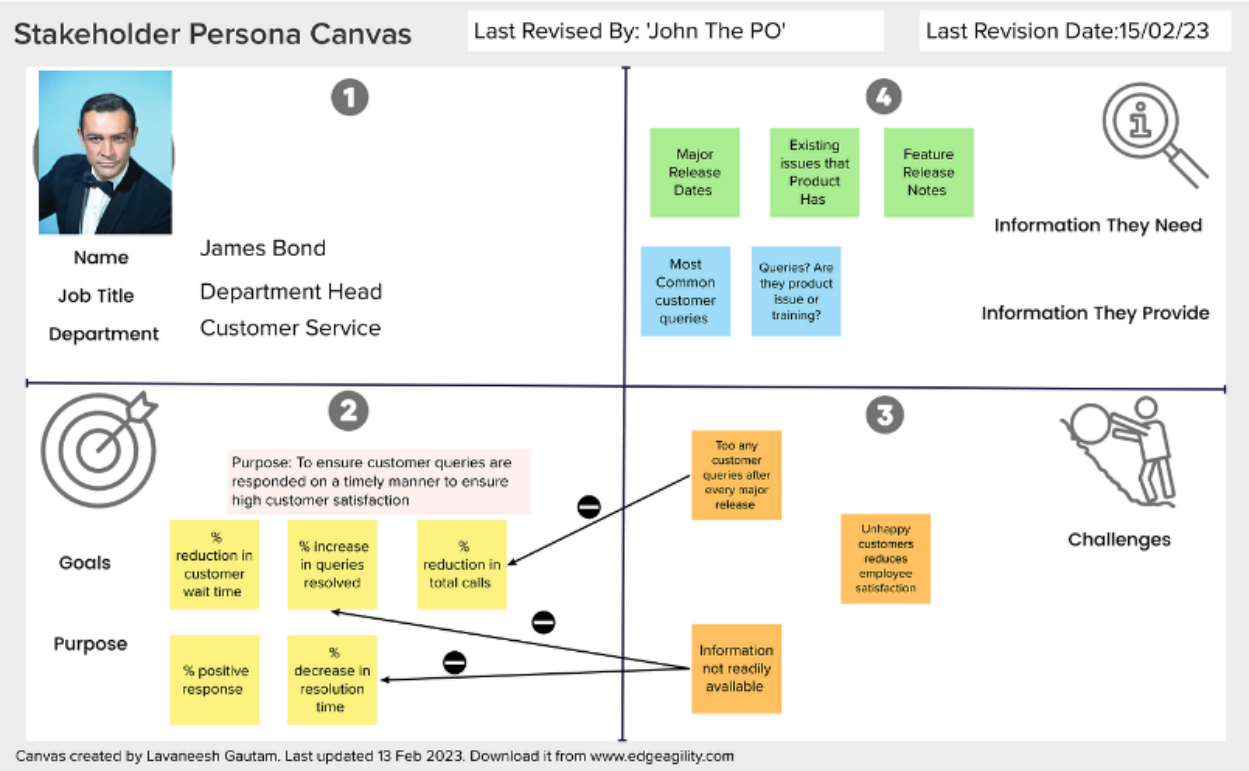

Figure 12: Stakeholder persona

- Suposiciones tácitas o conocimientos tribales
- Conflictos entre diferentes áreas o roles

Tecnicas principales

- **Entrevistas**: Abiertas o estructuradas. Escuchar mas de lo que hablas.
- **Workshops o talleres**: Sesiones colaborativas(por ejemplo, event storming), permite alinear a varias personas al mismo tiempo.
- **Observacion directa(etnografia lijera)**: Ver como los usuarios realmente interactuan con el sistema(con un sistema similar), detecta necesidades no mencionadas explicitamente.
- **Cuestionarios o encuestas**: Utiles para escalar informacion cuando hay muchos usuarios.
- **Analisis de documentos existentes**: Manuales, procesos actuales, formularios, bases de datos. Por ejemplo: ver como se hacia en papel para entender el sistema.
- **Prototipos / maquetas / wireframe**: Permiten validar ideas rapido y revelar requerimientos ocultos.
- **Historias de usuario y ejemplos concretos**: Pedirle al stakeholder que *cuente como haria algo*.

Buenas practicas

- Prepararse, conocer dominio y el contexto del negocio
- Confirmar lo entendido
- Incluir stakeholders diversos para evitar sesgos
- Documentar decisiones, no solo pedidos

Caracteristicas de requerimientos efectivos

- **Completo**: incluye toda la información necesaria para entenderlo
- **Correcto**: describe con precisión la necesidad real que el sistema debe satisfacer
- **Viable**: puede ser implementado dentro de las capacidades y limitaciones del sistema y del proyecto
- **Necesario**: realmente necesario para el negocio o el usuario
- **Priorizado**: tiene una importancia clara y definida
- **No-ambigüo**: no da lugar a múlti ples interpretaciones
- **Verificable (testable)**: cuenta con criterios claros de aceptación

  Nota: Check https://asana.com/es/resources/smart-goals

Analisis de requerimientos

Un requerimiento generalmente aparece como una entidad, va a haber un rol que se encargara de analisarlo. Involucra alcanzar una comprensión más precisa y rica de cada requerimiento y representar los requerimientos de diferentes maneras. Incluye la identificación de dependencias, conflictos y prioridades. Permite detectar requerimientos redundantes, innecesarios o faltantes. Técnicas:

- Modelos
- Diagramas de flujo
- Casos de uso
- Prototipos

Caracteristicas de un buen conjunto de requerimientos

Debemos poder ser flexible y ser capaces de arreglarlos durante el proceso.

- **Completo**: no están ausentes requerimientos o información necesaria
- **Consistente**: los requerimientos no deben contradecirse entre sí
- **Trazable**: un requerimiento debe poder ser rastreado a su origen y a su implementación
- **Modificable**: los requerimientos deben poder ser modificados y guardar un registro de los cambios

Validacion de requerimientos

Se realiza cada vez que se termina un sprint, al comienzo del desarrollo, etc.

Es el proceso de asegurar que los requerimientos reflejan correctamente las necesidades reales de los stakeholders. Responde a la pregunta: ¿Esto es lo que realmente necesitamos? Se realiza antes (y durante) el desarrollo para evitar retrabajos costosos.

Buenas practicas

- Validar con los usuarios correctos(no solo con el cliente interno)
- Incluir validacion en **Definition of Done**
- Validar requerimientos complejos con casos de uso o prototipos
- Documentar acuerdo(quien aprueba que y cuando)

Gestion de requerimientos

Conjunto de practicas y herramientas para organizar, restear, priorizar y controlar los cambios en los requerimientos(es muy comun que cambien). Debemos estar preparados y que no se nos escapen cosas. Su objetivo es mantener la alineacion entre lo que se necesita, lo que se contruye y lo que se entrega.

Trazabilidad de requerimientos

Nos lo van a pedir, muy importante. Debemos tener capacidad de rastrear cada requerimiento desde su origen hasta su imlplementacion, prueba y despliegue. Para esto se usan herramientas como Jira, Trello, Linear, ClickUp, etc. En algunos casos tambien puede ser importante la **trazabilidad inversa**(no muy usada): desde el codigo hasta el requerimiento que lo origino.

Priorizacion de requerimientos

Recomiendan utilizar un metodo ya existentes, los mas comunes son:

- **MoSCoW**: Must, Should, Could, Won't
- **Valor vs. Esfuerzo**: Matriz de priorizacion
- **Kano**: Satisfiers, Dissatisfiers, Delighters

  Nota: Una correcta priorización no es solo técnica: también política y estratégica

Gestion de cambios

Es inevitable que cambien los requerimientos, por ello, necesitamos tener un proceso claro. Hay que tener en cuenta:

- Quien puede solicitar cambios
- Como se analiza su impacto
- Quien lo aprueba

Vision alternativa: Requerimientos como hipotesis

En lugar de ver los requerimientos como verdades absolutos, considerarlos como hipotesis de negocio/producto a validar. Permite adoptar un enfoque mas cientifico al desarrollo de software. Se toma como una hipotesis a validar, esta validacion se realiza mediante la implementacion.

Si no funciona, lo decarto y pruebo otra hipotesis.

TL;DR

- Los requerimientos son fundamentales para el éxito de un proyecto de software
- Existen distintos tipos de requerimientos y formas de clasificarlos
- La gestión de stakeholders es clave para la extracción efectiva de requerimientos
- La trazabilidad, priorización y gestión de cambios son esenciales para mantener el control
- La validación de requerimientos es crucial para evitar retrabajos y asegurar que se cumplen las necesidades reales

User stories

Son una forma de trabajar requerimientos desde la colaboracion, son la unidad minima de trabajo en un marco de agile. Es un objetivo final y **no** una funcionalidad, la funcionalidad va a ser parte de como vamos a llegar a ese objetivo, vista desde la perspectiva del usuario final.

Son informales, explicacion general desde la perspectiva del usuario final o cliente. Consta de algunas sentencias que remarcan el resultado final. No contiene el detalle, los requerimientos son agregados despues una vez se priorice la historia.

Objetivos

- Foco en la mirada desde el usuario final
- Habilitan la colaboracion en el equipo de desarrollo
- Apuntan a crear soluciones creativas

Espectativas vs realidad

Muchas veces pasa que por cuestiones de negocios, se intentan meter user stories que no estaban ni escritas, entonces, surge incertidumbre al cambiar el user story y el scrum master debe poder adaptarse y decidir cuales stories son importantes para el proximo sprint.

Hay varias maneras de resolver un user story, entonces en la etapa de planificacion se debe llegar a un acuerdo en comun para decir que requerimientos y que recursos implica cada user story.

El scrum master debe poder manejar la reunion de planificacion y salir con un sprint backlog, traduciendo los user stories para planificar todo lo que van a hacer para el proximo sprint. Se encarga de facilitar todo y lo que no sea posible discutir se pasara para la siguiente reunion. Puede ocurrir que un user story sea muy grande, entonces se parte la gran user stoy en varias user stories, o dividirla en varias subtareas.

User stories exitosas

Presentan la siguiente estructura:

- COMO <rol>
- QUIERO <evento>
- PARA <funcionalidad>

Ejemplo:

- COMO Usuario web
- QUIERO Consultar tabla de pedidos
- PARA Saber el estado de todos mis pedidos